01001010 01000101 01000101 01010000 01001000 01010000

JeePHP Framework Developer Manual

Complete beginner guide: CRUD, packages, and secure backups

VERSION 1.0.0 / BLACK + WHITE / PHP 8.1+

Routes • Controllers • Views • Database • Models
Security • Migrations • Jobs • Packages • Google Drive

A practical manual based on the JeePHP source snapshot dated 21 October 2026.

How to use this book

This book explains JeePHP version 1.0.0 in beginner-friendly steps. Follow the customer CRUD chapter from start to finish to create, list, view, edit, and delete records. Other chapters explain the framework features around that example.

JeePHP is still evolving. The PHP code and commands in this manual are based on the source files available on 21 October 2026. A feature reaches Composer/Packagist only after its source is pushed, tagged, and published. Check the installed package version before expecting a new command to exist.

Contents

- 01 JeePHP in one minute
- 02 Requirements and installation
- 03 First run and welcome page
- 04 Project folders

- 05 Configuration with .duck
- 06 Routes, controllers, and views
- 07 Database helpers
- 08 Migrations
- 09 Complete customer CRUD application
- 10 Models and relationships
- 11 Avoiding N+1 queries
- 12 DataTables AJAX
- 13 Validation and security
- 14 Business logic and service providers
- 15 Jobs, queues, and cron scheduling
- 16 Mail configuration
- 17 Common helpers and AuthO/RolesO
- 18 Updating JeePHP with Git
- 19 Production and deployment
- 20 Troubleshooting
- 21 Optional package catalog
- 22 Database backups and Google Drive
- Appendix Command reference

01 JeePHP in one minute

JeePHP is a small PHP framework with short helper functions. A route sends a URL to a controller. The controller reads or writes data and loads a view. The database helpers use PDO prepared statements for values.

```
route('/', 'HomeController@index');


$customers = table_fetch('customers');


load_view('customers/index', ['customers' => $customers]);
```

Use the simplest API that fits the job: table helpers for quick table work, models for reusable business data and CRUD, and BusinessLogic classes when a workflow needs its own place.

02 Requirements and installation

What you need

- PHP 8.1 or later on the command line.
- Composer installed and available as composer.
- MySQL or MariaDB plus the PDO MySQL extension when database features are used.
- The mbstring extension for string validation.
- VS Code is optional; any text editor can edit the files. XAMPP is optional if standalone PHP is installed.

Install from Composer

Use this after the package is available on Packagist and your PHP version satisfies its requirements:

```
composer create-project sivasakthi/jeephp MyApp

cd MyApp

php artisan serve
```

If Composer reports that no version can be installed, check `php -v`, the required PHP extensions, and whether the latest JeePHP release has been published. Do not use --ignore-platform-reqs as a permanent fix.

Run from a local JeePHP source folder

When you already have the JeePHP source code, use its project generator:

```
php artisan create MyApp
cd MyApp
composer install
php artisan serve
```

On Windows, run `artisan.bat serve` if PHP is on PATH, or run the PHP executable directly, for example `C:\xampp\php\php.exe artisan serve`.

Where the app opens

```
http://127.0.0.1:8000
```

`php artisan serve` uses the PHP built-in development server. PHP is still required; JeePHP is not a compiler and does not create a standalone executable. Do not use the built-in server as a production web server.

03 First run and welcome page

A fresh project opens the JeePHP welcome screen at `/`. It uses the black-and-white terminal style and animated 0/1 background. The front controller is `public/index.php`; the default home route is in `routes/web.php`, and its controller loads `app/Views/home.php`.

To change the home page, update the route/controller/view files. Keep the web server document root pointed at `public/` so framework source, configuration, and application code are not directly exposed.

04 Project folders

```
app/
  Controllers/       Handle HTTP requests
  Models/            Database models and relationships
  Views/             HTML templates
  Helpers/           Application helper functions
  BusinessLogic/     Reusable workflows and business rules
  Providers/          Application startup hooks
  Jobs/               One-off and scheduled task classes
  Mail/               Reserved for mail classes/templates
routes/
  web.php             Browser routes
  api.php             API routes
config/                App, database, and mail settings
database/
  migrations/         Table/schema changes
  seeders/            Reserved for seed data scripts
bootstrap/app.php      Loads .duck and providers
```

```
public/index.php       Web entry point
src/JeePHP/            Framework internals
artisan                  Artisan-style command entry point
artisan.bat              Windows command launcher
bin/jeep                  Shared command implementation
```

App code belongs under `app/`. Framework internals belong under `src/JeePHP/`. Put browser routes in `routes/web.php`. `app/routes.php` is a compatibility loader; when upgrading an older project, move your existing custom routes into `routes/web.php` first.

Artisan-style CLI

JeePHP routes its existing framework commands through the Artisan-style php artisan entry point. Run it with no arguments, list, or --help to see the command list. All JeePHP commands shown in this book work through this entry point. On Windows, use artisan.bat. The legacy php jeep and jeep.bat launchers remain available.

```
php artisan
```

Generate common files

```
php artisan make:controller Customer
php artisan make:model Customer
php artisan make:logic RegisterCustomer
php artisan make:job DailyReport
```

05 Configuration with .duck

JeePHP uses `.duck` for local environment settings instead of `.env`. Composer copies `.duck.example` to `.duck` when a project is created. Keep `.duck` private; it is ignored by Git.

```
APP_ENV=local
APP_DEBUG=true
APP_NAME=MyApp
APP_URL=http://127.0.0.1:8000

DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=my_app
DB_USER=root
DB_PASS=
```

Create the database in MySQL/MariaDB, then set DB_NAME, DB_USER, and DB_PASS. The connection is opened when the first database helper is used. For production, set `APP_DEBUG=false` and use production credentials.

App-level settings live in `config/app.php`; DB settings are mapped in `config/database.php`; mail settings are mapped in `config/mail.php`. Config files read values parsed from `.duck` during bootstrap.

06 Routes, controllers, and views

Define simple routes

```
// routes/web.php
url_remove('.php');

route('/', 'HomeController@index');
route('/contact', 'ContactController@index');
route('/users/:id', 'UserController@show');
route('POST /users', 'UserController@store');
route('PUT /users/:id', 'UserController@update');
route('DELETE /users/:id', 'UserController@destroy');
```

A route without a method uses GET. JeePHP supports GET, POST, PUT, PATCH, DELETE, OPTIONS, and HEAD. A `:id` part captures one URL segment and is available as `$request->route('id')`. In a normal HTML form, use GET or POST; this framework snapshot does not document hidden method spoofing, so keep update/delete forms on POST routes where needed.

`url_remove('.php')` configures extensionless URLs. The router removes the suffix for matching routes. GET/HEAD requests to a `.php` URL redirect to the extensionless URL; generated URLs also remove the suffix.

Controller loads a view

```
<?php
namespace App\Controllers;

use JeePHP\Http\Request;

final class ContactController
{
    public function index(Request $request): void
    {
        load_view('contact', ['title' => 'Contact']);
    }
}
```

`load_view()` prints the view immediately. The controller does not need `ob_start()` or a return statement. Views are PHP files under `app/Views/`, such as `app/Views/contact.php`.

Pass a value into a view

```
// Controller
load_view('about', ['heading' => 'About us']);

<!-- app/Views/about.php -->
<h1>{heading}</h1>
```

JeePHP replaces `{heading}` and HTML-escapes the value. For more complex templates, use PHP directly and wrap output in `e($value)`. For repeated rows, use a repeat block or a PHP foreach loop.

07 Database helpers

Connect, fetch, filter

```
// Open the query builder for a table
$query = table_connect('customers');

// Fetch all rows (array of associative arrays)
$rows = table_fetch('customers');

// Choose columns, filter, sort, limit, and fetch
$rows = table_load('customers')
    ->columns('id', 'name', 'email')
    ->where('status', 'active')
    ->orderBy('name', 'ASC')
    ->limit(50)
    ->get();

$one = table_load('customers')->where('id', 1)->first();
$count = table_load('customers')->where('status', 'active')->count();
```

`table_connect()`, `table_load()`, `fetch_table()`, and `load_table()` return the query builder. `table_fetch()` runs the query immediately and returns rows. `where()` accepts a column, value, and optional operator; values are bound with PDO prepared statements. Use `columns()` for an explicit list. Table and column names are identifier-checked.

Insert, update, delete

```
$id = table_connect('customers')->insert([
    'name' => 'Siva',
    'email' => 'siva@example.com',
]);

table_connect('customers')
    ->where('id', $id)
    ->update(['name' => 'Sivasakthi']);

table_connect('customers')->where('id', $id)->delete();
```

Update and delete require a `where()` condition to prevent accidentally changing or deleting every row. Do not build SQL by joining user input into a query.

Show table data in a view

```
// Controller
fetch_table('customers')
    ->where('status', 'active')
```

```
    ->orderBy('id', 'DESC')
    ->limit(100)
    ->load_table('customers/index');


<!-- app/Views/customers/index.php -->
<h1>Customers</h1>
<table>
  {#customers}
  <tr><td>{name}</td><td>{email}</td></tr>
  {/customers}
</table>
```

`load_table()` passes rows using the table name as the view variable, then prints that view. It caps a result at 500 rows by default; choose a smaller `limit()` for real screens. For one row, scalar columns can also be accessed directly as `{column}`.

08 Migrations

A migration is a PHP file that creates or removes schema in a repeatable way. Run the CLI from the project root.

```
php artisan migrate:create customers
php artisan migrate
```

The command creates a timestamped file in `database/migrations/`. Open it and define the columns. `id` is created automatically unless you call `withoutId()`.

```
<?php
declare(strict_types=1);


use JeePHP\Database\Migration;


return new class implements Migration {
    public function up(\PDO $database): void
    {
        create_table('customers')
            ->stringColumn('name', 150, false)
            ->stringColumn('email', 190, false)
            ->stringColumn('phone', 30)
            ->timestamps()
            ->create();
    }


    public function down(\PDO $database): void
    {
        drop_table('customers');
    }
};
```

`column('name', 'email')` makes nullable VARCHAR(255) columns. Other helpers include `integer()`, `text()`, `decimal()`, `boolean()`, `stringColumn()`, and `timestamps()`.

Check the generated migration before running it. Rollback calls `down()` and can delete table data.

```
php artisan migrate:rollback
```

09 Complete customer CRUD application

CRUD means Create, Read, Update, and Delete. This example builds a small customer manager. First create the migration, then model, routes, controller, and views. Replace the sample fields if your database uses different names.

Step 1 Create the customers table

```
php artisan migrate:create customers
```

Edit the generated file as shown in Chapter 08. Include the fields `name`, `email`, and `phone`. Then run:

```
php artisan migrate
```

Step 2 Create the model

```
php artisan make:model Customer
```

Edit `app/Models/Customer.php`:

```
<?php
declare(strict_types=1);

namespace App\Models;

use JeePHP\Database\Model;

final class Customer extends Model
{
    protected static string $table = 'customers';

    protected array $fillable = ['name', 'email', 'phone'];
}
```

`$table` connects the model to the table. `$fillable` is the allow-list for fields accepted by `create()`, `insert_data()`, and `update()`. Do not add fields users must never control, such as admin flags, to a form allow-list without a reason.

Step 3 Add routes

Add these definitions to `routes/web.php`. Static routes come before `:id` routes for readability:

```
route('/customers', 'CustomerController@index');
route('/customers/create', 'CustomerController@create');
route('POST /customers', 'CustomerController@store');
route('/customers/:id/edit', 'CustomerController@edit');
route('POST /customers/:id/update', 'CustomerController@update');
route('POST /customers/:id/delete', 'CustomerController@destroy');
route('/customers/:id', 'CustomerController@show');
```

POST is used for actions that change data. All POST forms must include `<?= safe_form() ?>`; JeePHP checks the CSRF token before the controller runs.

Step 4 Write the controller

Create `app/Controllers/CustomerController.php`. This single controller demonstrates all five screens/actions. `validate()` returns an array of errors; the example re-renders the form with those errors instead of saving invalid data.

```
<?php
declare(strict_types=1);

namespace App\Controllers;

use App\Models\Customer;
use JeePHP\Http\Request;

final class CustomerController
{
    public function index(Request $request): void
    {
        $customers = Customer::query()->orderBy('id', 'DESC')->get();
        load_view('customers/index', ['customers' => $customers]);
    }

    public function create(Request $request): void
    {
        load_view('customers/form', ['customer' => null, 'errors' => []]);
    }

    public function store(Request $request): void
    {
        $values = $request->only('name', 'email', 'phone');
        $errors = validate($values, [
            'name' => 'required|min:2|max:100',
            'email' => 'required|email|max:190',
            'phone' => 'required|min:7|max:30',
        ]);
        if ($errors !== []) {
            http_response_code(422);
            load_view('customers/form', ['customer' => $values, 'errors' => $errors]);
            return;
        }

        Customer::create($values);
        redirect('customers');
    }
```

```
    public function show(Request $request): void
    {
        $customer = Customer::find((int) $request->route('id'));
        if ($customer === null) { http_response_code(404); echo 'Customer not found'; return; }
        load_view('customers/show', ['customer' => $customer]);
    }

    public function edit(Request $request): void
    {
        $customer = Customer::find((int) $request->route('id'));
        if ($customer === null) { http_response_code(404); echo 'Customer not found'; return; }
        load_view('customers/form', ['customer' => $customer, 'errors' => []]);
    }

    public function update(Request $request): void
    {
        $customer = Customer::find((int) $request->route('id'));
        if ($customer === null) { http_response_code(404); echo 'Customer not found'; return; }
        $values = $request->only('name', 'email', 'phone');
        $errors = validate($values, [
            'name' => 'required|min:2|max:100',
            'email' => 'required|email|max:190',
            'phone' => 'required|min:7|max:30',
        ]);
        if ($errors !== []) {
            http_response_code(422);
            load_view('customers/form', ['customer' => array_merge($customer->toArray(), $values),
'errors' => $errors]);
            return;
        }
        $customer->update($values);
        redirect('customers');
    }

    public function destroy(Request $request): void
    {
        $customer = Customer::find((int) $request->route('id'));
        if ($customer === null) { http_response_code(404); echo 'Customer not found'; return; }
        $customer->delete();
        redirect('customers');
    }
}
```

The `$request` argument is injected by the router. `only()` is useful because it limits which submitted fields are passed to the model. Always add authorization checks before allowing a user to view, edit, or delete another user's data.

Step 5 Build the customer list view

Create `app/Views/customers/index.php`:

```
<h1>Customers</h1>
<p><a href="<?= e(url('customers/create')) ?>">Add customer</a></p>
<table>
  <thead><tr><th>Name</th><th>Email</th><th>Phone</th><th>Actions</th></tr></thead>
  <tbody>
  <?php foreach ($customers as $customer): ?>
    <tr>
      <td><?= e($customer->name) ?></td>
      <td><?= e($customer->email) ?></td>
      <td><?= e($customer->phone) ?></td>
      <td>
        <a href="<?= e(url('customers/' . $customer->id)) ?>">View</a>
        <a href="<?= e(url('customers/' . $customer->id . '/edit')) ?>">Edit</a>
        <form method="post" action="<?= e(url('customers/' . $customer->id . '/delete')) ?>">
          <?= safe_form() ?>
          <button type="submit" onclick="return confirm('Delete this customer?')">Delete</button>
        </form>
      </td>
    </tr>
  <?php endforeach; ?>
  </tbody>
</table>
```

The view uses `e()` for database output so names/emails are escaped as HTML. The delete action is POST and includes the CSRF field. For real projects, style the form and add authentication/authorization.

Step 6 Create and edit form

Create `app/Views/customers/form.php`:

```
<?php
$isEdit = is_object($customer);
$id = $isEdit ? $customer->id : '';
$value = static function (string $key) use ($customer): string {
    if (is_array($customer)) return e($customer[$key] ?? '');
    return is_object($customer) ? e($customer->{$key}) : '';
};
?>
<h1><?= $isEdit ? 'Edit customer' : 'Add customer' ?></h1>
<?php foreach ($errors as $field => $messages): ?>
  <?php foreach ($messages as $message): ?><p><?= e($message) ?></p><?php endforeach; ?>
<?php endforeach; ?>
<form method="post" action="<?= e(url($isEdit ? 'customers/' . $id . '/update' : 'customers')) ?>">
  <?= safe_form() ?>
  <label>Name <input name="name" value="<?= $value('name') ?>" required></label>
```

```
  <label>Email <input name="email" type="email" value="<?= $value('email') ?>" required></label>
  <label>Phone <input name="phone" value="<?= $value('phone') ?>" required></label>
  <button type="submit">Save</button>
</form>
```

The `value()` closure handles a model during edit and an array when validation fails. HTML `required` is helpful for users, but server validation remains necessary.

Step 7 Create the detail view

Create `app/Views/customers/show.php`:

```
<h1><?= e($customer->name) ?></h1>
<p>Email: <?= e($customer->email) ?></p>
<p>Phone: <?= e($customer->phone) ?></p>
<a href="<?= e(url('customers')) ?>">Back to list</a>
```

Step 8 Test each CRUD action

1. Open /customers and confirm the list loads.
2. Choose Add customer, submit valid values, and confirm the new row appears.
3. Open View and Edit for a row. Change a value and save.
4. Try an invalid email and confirm the controller does not save it.
5. Delete a test row and confirm it disappears.
6. If a POST request returns 419, check that the form contains `<?= safe_form() ?>`.

10 Models and relationships

A model is a PHP class for a table. Generate one with `php artisan make:model Customer`, then set `$table` and `$fillable`. Use `Customer::find($id)`, `Customer::all()`, `Customer::create([...])`, `$model->update([...])`, `$model->delete()`, or `Customer::query()` for builder-based reads.

Model creation/update only accepts fields in `$fillable`. A model hides `password` from `toArray()` by default. Hash passwords before saving with `hash_password($password)` and verify them with `verify_password($plain, $hash)`.

Define a belongs-to relation

```
// app/Models/Post.php
public function author(): RelationDefinition
{
    return $this->belongsTo(User::class, 'user_id');
}

// Load posts and their authors in a batch
$posts = Post::with('author')->get();
```

For one-to-many, define `$this->hasMany(Post::class, 'user_id')` on User and call `User::with('posts')->get()`. Eager-loaded relations are available as `$post->author` or `$user->posts`.

11 Avoiding N+1 queries

N+1 happens when the app fetches a list, then runs one extra query for each row. For example, 100 posts plus 100 author lookups means about 101 queries. Eager loading

retrieves the parent rows and then fetches matching related rows in batched WHERE IN queries.

```
$posts = Post::with('author')->where('status', 1)->get();
foreach ($posts as $post) {
    echo e($post->author?->name);
}
```

JeePHP supports direct `belongsTo` and `hasMany` relationships here. Accessing a declared relation that was not included in `with()` throws an error instead of silently running a query per row. Nested loading such as `with('posts.author')` is not supported in this version. Arbitrary query-helper calls inside your own loops can still cause N+1; with `APP_DEBUG=true`, repeated identical SELECT statements are reported as possible warnings after five executions, and appear in the local Developer Console panel.

12 DataTables AJAX

For server-side DataTables, select allowed columns in PHP and pass the incoming Request to `ajax()`. JeePHP handles the usual draw/start/length/search/order inputs and returns a JSON response. The browser cannot choose arbitrary SQL column names.

```
use JeePHP\Http\Request;


public function load_data(Request $request): string
{
    $result = load_table('customers')
        ->where_column('id', 'name', 'email', 'phone')
        ->ajax($request);


    return json_data($result);
}
```

Define a GET route for the endpoint, then configure DataTables to call it with `serverSide: true`. The response is capped at 500 records per request. Use regular `data($query)` and `load_view()` for normal page rendering instead of the AJAX format.

13 Validation and security

Development Console tab

When the project runs in local/development mode with APP_DEBUG=true, JeePHP adds a Console tab at the bottom-right of HTML pages. Click it to review PHP exceptions, JavaScript errors, unhandled promises, console.error messages, and possible N+1 query warnings. PHP exceptions show the message, source file and line, stack trace, and a suggested next check. Suggestions are debugging hints; verify the fix in your application.

```
APP_ENV=development
APP_DEBUG=true
```

Set APP_ENV=production and APP_DEBUG=false before deploying. The Console panel and detailed exception page are disabled outside local/development debug mode. Keep debug mode private because error messages and stack traces can reveal source paths and application details.

CSRF tokens for forms

All state-changing POST, PUT, PATCH, and DELETE requests are protected by JeePHP security. Add `safe_form()` inside every HTML form that changes data. `csrf_field()` is an alias. JavaScript clients can send `csrf_token()` in the `X-CSRF-TOKEN` header.

```
<form method="post" action="<?= e(url('customers')) ?>">
    <?= safe_form() ?>
    <input name="name" required>
    <button type="submit">Save</button>
</form>
```

Validate on the server

```
$errors = validate($request->only('name', 'email'), [
    'name' => 'required|min:2|max:100',
    'email' => 'required|email',
]);
```

Supported rules include `required`, `nullable`, `email`, `url`, `numeric`, `integer`, `boolean`, `phone`, `date`, `min:n`, `max:n`, `confirmed`, `same:field`, `in:a,b`, `regex:pattern`, `unique:table,column[,ignoreId]`, and `exists:table,column`. Browser validation can be bypassed; validate every request in PHP too.

Security checklist

- Escape HTML output with e($value), including values read from the database.
- Use prepared query helpers and never concatenate user input into SQL.
- Limit mass assignment with model $fillable and Request::only().
- Hash passwords with hash_password(); never save plain text passwords.
- Keep real secrets in .duck and do not commit it.
- Set APP_DEBUG=false in production.
- Add authentication and authorization checks; CSRF protection alone is not access control.
- Back up the database and review destructive migrations before running or rolling back.

The `<safe>` syntax described earlier in discussion is not implemented. Use `safe_form()` in HTML views for the current built-in CSRF protection.

For APIO routes under `/api/`, a valid `Authorization: Bearer TOKEN` can replace the browser CSRF token. The API controller must still call `api_require()` and name the required ability. An invalid token is not permission; never leave a data-changing API route without an authorization check.

14 Business logic and service providers

Business logic

Put reusable application workflows in `app/BusinessLogic/`. Generate a starter class:

```
php artisan make:logic RegisterCustomer
```

The generated class has a `handle()` method. A controller can construct and call it. Move rules that need reuse or coordinate multiple models out of a long controller method.

Service providers

`config/app.php` lists provider classes. JeePHP calls `register()` on all providers, then calls `boot()` on all providers. Use `app/Providers/AppServiceProvider.php` for application startup registration, not for every request-specific operation.

```
namespace App\Providers;

use JeePHP\Foundation\ServiceProvider;

final class AppServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // Register application services.
    }

    public function boot(): void
    {
        // Run setup after providers have registered.
    }
}
```

15 Jobs, queues, and cron scheduling

A simple one-time job can run synchronously. Generate a class from the project root:

```
php artisan make:job DailyReport
php artisan job:run DailyReport
```

For background work, activate QueueO, run its migration, and create a QueueableJob class:

```
php artisan activate QueueO
composer dump-autoload
php artisan migrate
php artisan make:job SendReceipt
```

Dispatch it from a controller/business-logic class, then keep a worker running:

```
queue_dispatch('SendReceipt', ['order_id' => 42]);

php artisan queue:work
php artisan queue:failed
php artisan queue:retry JOB_ID
```

QueueO saves jobs in MySQL/MariaDB, reserves one job at a time, retries failures up to the configured limit, and keeps exhausted jobs for inspection. `php artisan queue:work --once` processes at most one ready job. Use a process manager for a production worker.

Recurring schedules

Activate ScheduleO, add a task to `config/schedule.php`, and run `php artisan schedule:run` once a minute from Linux cron or Windows Task Scheduler. `every_seconds`

supports values from 60 seconds to one year. Due jobs are queued if QueueO is active; otherwise they run inside the scheduler command.

```
return [
    ['job' => 'DailyReport', 'every_seconds' => 86400, 'payload' => []],
];
```

When scheduled tasks go to QueueO, keep a queue worker running to process them.

16 Mail configuration

MailO sends email over SMTP using PHPMailer. Activate the package and install its Composer dependency:

```
php artisan activate MailO
composer update
```

Set SMTP values in `.duck`; keep the real password out of Git:

```
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=your-user
MAIL_PASSWORD=your-secret
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@example.com
MAIL_FROM_NAME=MyApp
```

Create a PHP template under `app/Views/emails/`, escape user values with `e()`, and send it with `send_mail($to, $subject, $viewName, $data)`. Test with `php artisan mail:test user@example.com`. AuthO password reset and email verification need MailO to deliver their messages.

17 Common helpers

```
table_connect('customers')       // start a query builder
table_fetch('customers')         // fetch all rows
table_load('customers')          // start a query builder
fetch_table('customers')         // start a query builder
load_table('customers')          // start a query builder
create_table('customers')        // start migration schema builder
load_view('customers/list', $data) // print a view
view('customers/list', $data)   // lower-level view helper
e($value)                        // HTML escape
url('customers')                 // build an application URL
asset('app.css')                 // build a public asset URL
redirect('customers')            // redirect to an app URL
session('user_id')               // read session value
flash('notice', 'Saved')          // store one-time message
hash_password($password)         // password hash
verify_password($plain, $hash)   // verify password
safe_form()                      // hidden CSRF field
validate($input, $rules)         // server-side validation
```

Add app-specific reusable functions in `app/Helpers/helpers.php`; Composer loads that file automatically.

The validator supports `required`, `nullable`, `email`, `url`, `numeric`, `integer`, `boolean`, `phone`, `date`, `min:n`, `max:n`, `confirmed`, `same:field`, `in:a,b`, `regex:pattern`, `unique:table,column[,ignoreId]`, and `exists:table,column`. Database rules use checked identifiers and prepared values.

Optional packages: AuthO and RolesO

JeePHP ships two optional first-party scaffolds. Activate only the features your application uses. Run commands from the project root:

```
php artisan activate AuthO
php artisan activate RolesO
php artisan --version
```

Activation adds reviewed files under `app/`, adds routes to `routes/web.php`, adds helper functions to `app/Helpers/helpers.php`, and creates a migration in `database/migrations/`. Existing app files are not overwritten; if a generated file already exists, activation stops so you can resolve the conflict. Run `composer dump-autoload` and then `php artisan migrate` to register helpers and create the tables.

AuthO authentication

AuthO adds registration, sign-in, account, and sign-out routes at `/auth/register`, `/auth/login`, `/auth/account`, and POST `/auth/logout`. Passwords must be 12–72 bytes; it hashes them with PHP `password_hash()` and verifies with `password_verify()`. It uses JeePHP’s global CSRF check, rotates the session ID and CSRF token after sign-in, returns a generic invalid-credentials message, and applies signup/sign-in attempt limits. Missing accounts also use a dummy password hash during verification. Account pages require a signed-in session.

```
auth_check()                // is a user signed in?
$user = auth_user();         // signed-in user fields (no password hash)
auth_logout();               // clear the authenticated session
```

AuthO adds password reset routes. Email verification can be enabled in `.duck` with `AUTH_REQUIRE_EMAIL_VERIFICATION=true`; install MailO to send the verification link. The package is a practical baseline, not a guarantee against every attack. Deploy over HTTPS, use a production database account, monitor login abuse, and review your application’s authorization rules.

RolesO roles and permissions

RolesO creates `roles`, `permissions`, `user_roles`, and `role_permissions` tables, a protected `/roles` management page, and helpers for assigning roles and checking permissions. The management page requires a signed-in account that already has the `roles.manage` permission. After AuthO and RolesO migrations have run, bootstrap the first manager from the CLI using an existing AuthO user ID:

```
php artisan role:assign 1 admin
php artisan permission:grant admin roles.manage
```

Replace `1` with the intended account ID. Sign in as that user and open `/roles`; create further roles and permissions there. The same helpers are available in application code:

```
assign_role($userId, 'editor');
grant_permission('editor', 'posts.publish');
```

```
if (user_has_role($userId, 'editor')) { /* ... */ }
if (user_can('posts.publish', $userId)) { /* allow action */ }
```

Call `user_can()` or `user_has_role()` in every protected controller/action. Hiding a button in a view does not protect its route. Run `php artisan activate RolesO` only after deciding which authenticated users should administer permissions.

18 Updating JeePHP with Git

When you change the framework source, update the Git repository so the next install can receive it. Work from the JeePHP repository folder in the VS Code terminal:

```
git status
git add .
git commit -m "Describe the JeePHP update"
git push origin main
```

Review `git status` before staging. Do not commit `.duck`, database passwords, Google OAuth files, API secrets, or customer data. To publish a Composer release, create and push a new immutable version tag and ensure Packagist sees it. The earlier v1.0.0 and v1.1.0 tags are already used, so publish the next source release under a new tag such as v1.2.0. Existing Composer projects do not change when Git main changes; update them through the newly published release.

19 Production and deployment

- Point Apache/Nginx document root to the project public/ directory.
- Enable Apache mod_rewrite and allow .htaccess overrides if you depend on the included rewrite rules.
- Use HTTPS, production credentials, APP_DEBUG=false, and least-privilege database accounts.
- Keep .duck, app/, config/, database/, and src/ outside the public web root.
- Run Composer install for the deployment environment and apply reviewed migrations.
- Back up database data before schema changes.

The project root index.php provides a local XAMPP fallback, while `.htaccess` routes clean URLs into public/. For XAMPP, enable mod_rewrite and allow .htaccess overrides. Production servers should point their document root directly to public/.

20 Troubleshooting

XAMPP opens app/Views instead of the welcome page

Open http://localhost/MyApp/; never open app/Views or a view file as a URL. If the app/Views folder or a directory listing appears, Apache is ignoring the project .htaccess. In C:/xampp/apache/conf/httpd.conf, enable LoadModule rewrite_module modules/mod_rewrite.so and set the C:/xampp/htdocs Directory block to AllowOverride All, then restart Apache. The root .htaccess routes / to the welcome page and blocks framework source folders. Root index.php can show the welcome page without rewrites, but clean routes and source-folder protection need the Apache settings.

Composer says PHP version is not supported

Run `php -v` and check the PHP executable Composer is using. JeePHP source requires PHP 8.1 or later. If using XAMPP on Windows, make sure Composer is configured to use the intended XAMPP PHP executable.

Database connection error

Check that the MySQL service is running, the database exists, DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASS in `.duck` are correct, and PDO MySQL is enabled. The initial landing page does not need a database connection.

POST form returns 419

Include `<?= safe_form() ?>` inside the form. Confirm the form is submitted using POST and session cookies are enabled.

A model field is ignored

Add the field name to the model `$fillable` list and confirm the database table contains that column. Use `$request->only()` to pass only fields the user is allowed to change.

N+1 exception when reading a relation

Load that relationship with `Model::with('relationName')->get()` before accessing it. This version supports only direct `belongsTo` and `hasMany` relationships.

New command missing after Composer install

Check the installed JeePHP release. New local Git commits are not part of an already-published Composer archive until a release is tagged/published. For a local development project, test from the updated source repository.

No email is sent

Confirm MailO is active, run `composer update`, check the SMTP values and sender address in `.duck`, and use `php artisan mail:test your-address@example.com`. Check the provider’s SMTP port and TLS settings.

21 Optional package catalog

Run `php artisan package:list` to review available features. Activation copies scaffold files and writes an active marker. It does not run migrations. Existing application files are never overwritten. After activation, follow the CLI next steps: run `composer update` if composer.json changed, run `composer dump-autoload`, inspect any generated migration, and then run `php artisan migrate`.

Package dependencies: RolesO needs AuthO; UploadO needs StorageO; CrudO needs ValidateO; MailO installs PHPMailer; BackupO requires PHP cURL, OpenSSL, and zlib.

Feature list

- AuthO: registration, sign-in, account, sign-out, password reset, and optional email verification.
- RolesO: role and permission tables, a management page, and `user_can()` helpers.
- MailO: `send_mail()` over SMTP, using PHPMailer.
- QueueO: `queue_dispatch()`, a queue worker, retries, failed-job inspection, and retry command.
- ScheduleO: `config/schedule.php` and `php artisan schedule:run`.
- TestO: `php artisan make:test` and a small custom test runner (`php artisan test`), not PHPUnit.
- UploadO: `upload_file()` validates PHP upload status, byte limit, detected MIME, and uses random filenames.

- CacheO: `cache_remember()` and `cache_forget()` for local file cache entries.
- APIO: API tokens with expiry and abilities; token hashes are stored in the database.
- CrudO: `php artisan make:crud Product name,price,description` generates a starter CRUD module.
- ValidateO: detailed validation rules; `validate()` itself is part of the framework core.
- LogO: `app_log()` writes JSON logs and redacts secret-like context keys.
- StorageO: `store_file()`, `read_file()`, `delete_file()`, and `storage_url()`; private storage is the default.
- BackupO: MySQL/MariaDB local backup, encryption, Google Drive upload, restore, and retention.
- LocaleO: `set_locale()` and `trans()` with English and Tamil example messages.

API tokens

```
php artisan activate APIO
composer dump-autoload
php artisan migrate
php artisan api:token USER_ID mobile-app users.read,users.write
```

Copy the token printed by the command; it is only shown once. Send it only over HTTPS in the Authorization header. In each API controller, call `api_require('users.read')` and return if it gives null. `*` means every ability, so avoid it unless required. Revoke a token with `php artisan api:revoke TOKEN_ID USER_ID`.

Generate a CRUD module

```
php artisan activate ValidateO
php artisan activate CrudO
composer dump-autoload
php artisan make:crud Product name,price,description
php artisan migrate
```

Review the generated routes, migration, controller, validation, and views. Add authentication and authorization before using the module with private or production data.

22 Database backups and Google Drive

BackupO backs up the configured MySQL/MariaDB database to local compressed SQL files. It can encrypt each backup with AES-256-GCM before uploading it to a JeePHP folder in Google Drive.

Set up BackupO

```
php artisan activate BackupO
composer update
php artisan backup:key-generate
```

Copy the generated key into `.duck` as `BACKUP_ENCRYPTION_KEY=...`. Keep another copy offline and outside this project. Losing it means encrypted backups cannot be restored. BackupO requires PHP cURL, OpenSSL, and zlib to be enabled for the same PHP executable used by Composer and JeePHP.

Connect Google Drive

- In Google Cloud Console, create a project and enable the Google Drive API.

- Create an OAuth client with application type Desktop app, then download its JSON file.
- From the JeePHP project root, run `php artisan drive:connect path/to/oauth-client.json`.
- Copy the URL printed by the CLI into a browser, approve access, and wait for the local callback.
- Run `php artisan drive:status` to confirm the local authorization file exists.

JeePHP checks OAuth state and uses PKCE. It stores the OAuth client and refresh token under ignored `storage/private/` with restrictive local file permissions. The requested Drive scope is limited to files JeePHP creates in its own backup folder. Never commit the OAuth JSON or token files.

Create and list backups

```
php artisan backup:database
php artisan backup:database --drive
php artisan backup:list
php artisan backup:list --drive
```

With BACKUP_ENCRYPTION_KEY set, local files are encrypted too. Without a key, local files are compressed SQL and must be treated as sensitive. Google Drive uploads always require encryption. A Drive upload also keeps the local backup copy.

Restore or remove old backups

```
php artisan backup:restore BACKUP_FILE.sql.gz.enc --force
php artisan backup:restore --drive DRIVE_FILE_ID --force
php artisan backup:clean --keep=7
php artisan backup:clean --keep=7 --drive
php artisan drive:disconnect
```

Before restoring, confirm `DB_NAME` in `.duck` and make a separate backup of current data. The CLI asks you to type RESTORE because matching tables and their rows are dropped and recreated. It does not remove extra tables absent from the dump. Drive cleanup also removes local backups past the retention count and asks you to type DELETE before removing remote files. Disconnect removes local OAuth files and attempts remote token revocation.

Backup coverage and limits

This version exports MySQL/MariaDB base-table definitions, rows, and views. InnoDB tables use a consistent snapshot; nontransactional tables may change during the dump. Triggers, stored procedures/functions, scheduled database events, database users/grants, and application files are not included. The database must already exist. The backup user needs table read access; restore needs create, drop, and insert access. Test a restore using a disposable database before relying on a production backup.

Troubleshooting Drive setup

Check that the Drive API is enabled, the OAuth JSON is for a Desktop app, the Google user can approve access, and PHP cURL/OpenSSL are enabled. Re-run `php artisan drive:connect path/to/oauth-client.json`. A missing or incorrect encryption key blocks encrypted backup and restore. If remote revocation cannot be confirmed, revoke JeePHP from Google Account security settings.

Appendix Command reference

```
php artisan --version
php artisan serve
php artisan migrate
php artisan package:list
php artisan activate AuthO
php artisan activate RolesO
php artisan activate BackupO
php artisan backup:key-generate
php artisan drive:connect oauth-client.json
php artisan backup:database --drive
php artisan backup:list --drive
php artisan backup:restore --drive FILE_ID --force
php artisan backup:clean --keep=7 --drive
php artisan role:assign USER_ID admin
php artisan permission:grant admin roles.manage
```

Windows command alternatives: use artisan.bat instead of php artisan when PHP is available on PATH. The earlier php jeep and jeep.bat launchers remain supported.